

1.1.1. Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:
 m is the mass of the object (in kilograms).
 v is the velocity of the object (in meters per second).

- Input Format:**
- A single floating-point number representing the mass of the object in kilograms.
 - A single floating-point number representing the velocity of the object in meters per second.

- Output Format:**
- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

calculate...

1

mass=float(input())

2

velocity=float(input())

3

momentum = mass*velocity

4

print(f"momentum:.2f}kgm/s")

Average time

0.004 s

4.25 ms

Maximum time

0.005 s

5.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

4 ms

Debug

Expected output

Actual output

5.0

5.0

10.0

10.0

50.00kgm/s

50.00kgm/s

Test case 2

5 ms

Terminal

Test cases

Prev

Reset

Submit

Next

1.1.2. Conditional Calculation Based on the Number of Digits 15:22

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:
 $1 \leq n \leq 999$

Input Format:
• The input consists of a single integer n .

Output Format:
• If n is a single-digit number, print its square.
• If n is a two-digit number, print its square root (rounded to two decimal places).
• If n is a three-digit number, print its cube root (rounded to two decimal places).
• Else print "Invalid".

Sample Test Cases +

condition... Submit

1 import math
2 num=int(input())
3 if 1<= num <= 9:
4 print(num**2)
5 elif 10<= num <=99:
6 print(f"{math.sqrt(num):.2f}")
7 elif 100<=num<=999:
8 print(f"{math.cbrt(num):.2f}")
9 else:
10 print("Invalid")
11

Average time
0.003 s
2.86 ms

Maximum time
0.004 s
4.00 ms

4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 4 ms Debug

Expected output

Actual output

99

8181

Test case 2 2 ms

Test case 3 2 ms

Terminal

Test cases

1.1.3. Days between Two Dates

04:17

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

+

Explorer

noOfDay...

⌂

Submit

Debugger

```
1 from datetime import date
2
3 d1 = input().split('-')
4 d2 = input().split('-')
5
6 date1 = date(int(d1[0]), int(d1[1]), int(d1[2]))
7 date2 = date(int(d2[0]), int(d2[1]), int(d2[2]))
8
9 diff = date2 - date1
10
11 print(diff.days)
```

Average time

0.006 s

6.25 ms

Maximum time

0.009 s

9.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 9 ms

Debug

⌵

Expected output

2014-07-02

2014-11-02

123

Actual output

2014-07-02

2014-11-02

123

Test case 2 6 ms

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.1.4. Reverse a Number 01:30 AA 🌙 🔗 🔗 —

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases +

reverseN... 🔍 Submit

```

1 num=int(input())
2 reversed_num= int(str(num)[::-1])
3 print(reversed_num)

```

Average time
0.003 s
2.80 ms

Maximum time
0.004 s
4.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 4 ms 🐞 Debug ☰ 📄

Expected output	Actual output
5367	5367
7635	7635

✓ Test case 2 3 ms

🔍 Terminal ☰ Test cases

1.1.5. Multiplication Table 01:37

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases +

multipl...

1 num = int(input())

2 for i in range(1,11):

3 print(f"{num} x {i} = {num*i}")

Submit

Average time0.003 s3.25 ms

Maximum time0.005 s5.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 15 ms

Debug

Expected output

Actual output

8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

Terminal

Test cases

1.2.1. Pass or Fail

View Meetings

03:25

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

passorFa...

Submit

```

1 n = int(input())
2 marks = list(map(int, input().split()))
3 if any(mark < 40 for mark in marks):
4     print("Fail")
5 else:
6     percentage = sum(marks)/n
7     if percentage > 75:
8         grade = "Distinction"
9     elif percentage >= 60:
10        grade = "First Division"
11    elif percentage >= 50:
12        grade = "Second Division"
13    else:

```

Average time

0.005 s

4.90 ms

Maximum time

0.007 s

7.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 7 ms Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2 6 ms

Terminal

Test cases

Prev Reset Submit Next

1.2.2. Fibonacci Series 29.50

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases +

fibonacci...

```
10 # print(fibonacci(i), end=" ")
11 def fibonacci(n):
12     if n == 0:
13         return 0
14     elif n == 1:
15         return 1
16     else:
17         return fibonacci(n-1) + fibonacci(n-2)
18
19 n = int(input())
20
21 for i in range(n):
22     print(fibonacci(i), end=" ")
```

Average time0.009 s9.50 msMaximum time0.021 s21.00 ms

2 out of 2 shown test case(s) passed2 out of 2 hidden test case(s) passed

Test case 18 msDebug

Expected output50 1 1 2 3Actual output50 1 1 2 3

Test case 23 ms

Terminal

Test cases

Submit

Debugger

1.2.3. Pattern - 1

00:35



Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases



rightangl...



Submit

Explorer

```
1 a=int(input())
2 for i in range(1,a+1):
3     print("*"*i)
```

Debugger

Average time

0.003 s

3.00 ms



Maximum time

0.004 s

4.00 ms



2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 4 ms

Debug



Expected output

5

* *

* * *

* * * *

* * * * *

* * * * * *

Actual output

5

* *

* * *

* * * *

* * * * *

* * * * * *

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.2.4. Pattern - 2 00:45

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

```
numberP...
1 a=int(input())
2 for i in range(1,a+1):
3     for j in range(1,i+1):
4         print(j,end=" ")
5     print()
```

Average time 0.003 s 3.00 ms Maximum time 0.004 s 4.00 ms

2 out of 2 shown test case(s) passed 2 out of 2 hidden test case(s) passed

Test case 1 4 ms Debug	
Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Terminal Test cases